DATA STRUCTURES

CO4 Notes: Searching, Hashing and Sorting (Unit IV)

Course Code: PBC2TE302 | PBCA | Sem III

Course Outcome (CO4)

Understanding different searching and sorting algorithms, and their computational efficiencies.

Topics covered in these notes

- Searching techniques: linear search and binary search
- Hashing: direct-address tables, hash tables and hash functions
- Collision and open addressing: linear probing, quadratic probing, double hashing; chaining
- Perfect hashing
- Sorting techniques: bubble, selection, insertion and merge sort
- Time complexity analysis and comparison of sorting methods
- Quick revision and practice questions

1. Searching Techniques

Searching is the process of finding the location of a given element (called the **key**) in a collection of data. If the key is found the search is **successful**, otherwise it is **unsuccessful**. The two basic methods are linear search and binary search.

1.1 Linear search (sequential search)

The key is compared with **every element one by one**, starting from the first element, until it is found or the end of the list is reached. The list **need not be sorted**.

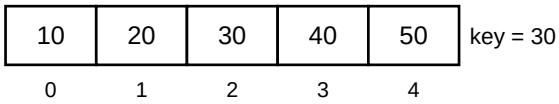
Algorithm: LINEAR_SEARCH(A, n, key)

Step 1: Set i = 0.

Step 2: Repeat while i < n : if A[i] == key, print "Found at position i" and stop; otherwise i = i + 1.

Step 3: If the loop ends, print "Key not found". Stop.

Example: search key 30 in [10, 20, 30, 40, 50]



Step	Compare	Result
1	A[0] = 10 with 30	not equal, move ahead
2	A[1] = 20 with 30	not equal, move ahead
3	A[2] = 30 with 30	**equal -> found at index 2**

```
int linear_search(int a[], int n, int key) {
    int i;
    for (i = 0; i < n; i++)
        if (a[i] == key) return i;     // found, return the position
    return -1;                          // not found
}
```

Case	Time complexity
Best case (key is the first element)	O(1)
Average case	O(n)
Worst case (key is last or absent)	O(n)

- **Advantages:** simple, works on unsorted data and on linked lists.
- **Disadvantage:** slow for large data because every element may be checked.

1.2 Binary search

Binary search works only on a **sorted list**. It compares the key with the **middle element**. If equal, the search is over. If the key is smaller, the search continues only in the **left half**; if larger, only in the **right half**. In each step half of the elements are discarded (divide and conquer).

Algorithm: BINARY_SEARCH(A, n, key)

Step 1: Set low = 0, high = n - 1.

Step 2: Repeat while low <= high :

(a) mid = (low + high) / 2

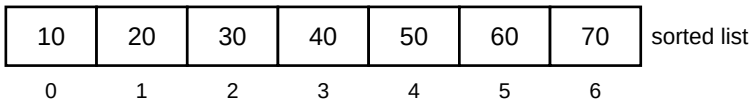
(b) If A[mid] == key, print "Found at mid" and stop.

(c) If key < A[mid], set high = mid - 1 (search left half).

(d) Else set low = mid + 1 (search right half).

Step 3: If the loop ends, print "Key not found". Stop.

Example: search key **60** in the sorted list [10, 20, 30, 40, 50, 60, 70] (indices 0 to 6)



Step	low	high	mid = (low+high)/2	A[mid]	Decision
1	0	6	3	40	60 > 40, so low = mid + 1 = 4
2	4	6	5	60	60 = 60 -> **found at index 5**

Unsuccessful search example: key = 35. Step 1: mid = 3, A[3] = 40, 35 < 40 so high = 2. Step 2: low = 0, high = 2, mid = 1, A[1] = 20, 35 > 20 so low = 2. Step 3: low = 2, high = 2, mid = 2, A[2] = 30, 35 > 30 so low = 3. Now low > high, so the key is **not found**.

```
int binary_search(int a[], int n, int key) {
    int low = 0, high = n - 1, mid;
    while (low <= high) {
        mid = (low + high) / 2;
        if (a[mid] == key)      return mid;
        else if (key < a[mid])  high = mid - 1;
        else                    low  = mid + 1;
    }
    return -1;
}
```

Case	Time complexity
Best case (key at the middle)	O(1)
Average and worst case	O(log n)

For n = 1000 elements, linear search may need 1000 comparisons but binary search needs only about 10 ($\log_2 1000$ is about 10).

Linear Search	Binary Search
Works on sorted and unsorted data	Works only on sorted data
Compares elements one by one	Compares with the middle element and halves the list
Time: O(n)	Time: O(log n)
Simple, small code	Slightly complex
Can be used on linked list easily	Needs random access (array)
Best for small lists	Best for large sorted lists

2. Hashing

Hashing is a technique to store and search data in **almost constant time O(1)**. Instead of searching through the data, a **hash function** converts the key directly into an **address (index)** of a table where the data is kept. The table is called a **hash table**.

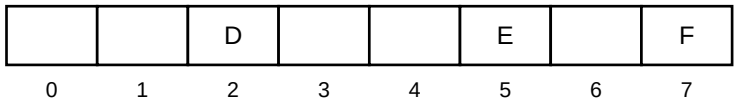
Key terms

- **Key:** the value used to identify the data (for example roll number).
- **Hash function h(k):** a formula that maps a key k to an index of the table.
- **Hash table:** an array in which data is stored at the index given by the hash function.
- **Collision:** when two different keys give the same index.
- **Load factor** = number of keys stored / size of table.

2.1 Direct-address tables

When the keys are small integers from a small range (the **universe** U = {0, 1, ..., m-1}), we can use an array T of size m and store the element with key k **directly at T[k]**. No hash function is needed.

- **Search, insert and delete take O(1) time.**
- **No collisions** because each key has its own slot.
- **Disadvantage:** if the range of keys is huge (for example 10-digit phone numbers) but only a few keys are actually used, a lot of memory is wasted. Hash tables solve this.



2.2 Hash tables and hash functions

A hash table of size m uses a hash function h(k) which gives a value from 0 to m-1. Common hash functions:

Method	Formula / idea	Example (table size 10)
Division method	h(k) = k mod m	h(27) = 27 mod 10 = 7
Mid-square method	Square the key and take the middle digits as the index	k = 25, 25^2 = 625 -> middle digit 2 -> index 2
Folding method	Divide the key into parts, add them and take mod m	k = 12345 -> 12 + 34 + 5 = 51 -> 51 mod 10 = 1

A good hash function should

- be easy and fast to compute,
- distribute the keys uniformly over the table, and
- produce as few collisions as possible.

2.3 Collision and collision resolution

Collision occurs when two or more different keys are mapped to the **same index** by the hash function. Example: with h(k) = k mod 10, the keys 23 and 43 both give index 3. Collisions must be resolved. There are two main approaches:

Method	Idea
Chaining (open hashing)	Every slot of the table holds a linked list; all keys having the same index are added to that list.
Open addressing (closed hashing)	All keys are stored inside the table itself. If the slot is occupied, another empty slot is searched using a probing sequence: linear probing, quadratic probing or double hashing.

(a) Separate chaining

Example (h(k) = k mod 5): keys 12, 17, 22, 8, 3

Index	Chain
0	NULL
1	NULL
2	12 -> 17 -> 22 -> NULL
3	8 -> 3 -> NULL
4	NULL

(b) Open addressing: Linear probing

If the slot h(k) is occupied, check the **next slots one by one** in a circular manner:

Probe sequence
• **index = (h(k) + i) mod m**, for i = 0, 1, 2, 3, ...
• i.e. try h(k), h(k)+1, h(k)+2 ... until an empty slot is found.

Example: table size 10, h(k) = k mod 10, insert 23, 43, 13, 27, 37 using linear probing.

Key	h(k)	Probing	Stored at
23	3	slot 3 is empty	3
43	3	slot 3 full -> slot 4 empty	4
13	3	slots 3, 4 full -> slot 5 empty	5
27	7	slot 7 is empty	7
37	7	slot 7 full -> slot 8 empty	8

Index	0	1	2	3	4	5	6	7	8	9
Key				23	43	13		27	37	

Drawback - primary clustering: keys form long continuous blocks (like 23, 43, 13) so searching and inserting become slower.

(c) Open addressing: Quadratic probing

To reduce clustering, quadratic probing jumps by **squares** of the step number:

Probe sequence
• **index = (h(k) + i^2) mod m**, for i = 0, 1, 2, 3, ...
• i.e. try h(k), h(k)+1, h(k)+4, h(k)+9, h(k)+16 ... (mod m).

Example: same keys and table (size 10): 23, 43, 13, 27, 37.

Key	h(k)	Probing steps	Stored at
23	3	slot 3 empty	3
43	3	i=0: slot 3 full; i=1: (3+1) = 4 empty	4
13	3	i=0: 3 full; i=1: 4 full; i=2: (3+4) = 7 empty	7
27	7	i=0: 7 full; i=1: (7+1) = 8 empty	8
37	7	i=0: 7 full; i=1: 8 full; i=2: (7+4) = 11 mod 10 = 1 empty	1

Index	0	1	2	3	4	5	6	7	8	9
Key		37		23	43			13	27	

Keys are spread more evenly than in linear probing. **Drawback - secondary clustering:** keys with the same starting index follow the same probe sequence.

```
#define M 10
int table[M];                        // initialise all to -1 (empty)

void insert_linear(int key) {
    int i = 0, idx;
    while (i < M) {
        idx = (key % M + i) % M;              // for quadratic: (key % M + i*i) % M
        if (table[idx] == -1) { table[idx] = key; return; }
        i++;
    }
    printf("Table is full");
}
```

(d) Double hashing

Uses a second hash function to decide the step size: **index = (h1(k) + i x h2(k)) mod m**. For example h1(k) = k mod 10 and h2(k) = 7 - (k mod 7). It removes both primary and secondary clustering.

Linear probing	Quadratic probing	Double hashing
Step size is 1	Step size is i^2	Step size from second hash function
Primary clustering	Secondary clustering	Least clustering
Easiest to implement	Moderate	Slightly complex

Note on deletion in open addressing: a deleted slot must be marked as "deleted" (tombstone) and not simply made empty, otherwise the search for other keys may stop early.

2.4 Perfect hashing

Perfect hashing is a method that gives **worst-case O(1) search time** with **no collisions at all**. It works when the **set of keys is fixed (static)** and known in advance, for example the reserved words of a programming language.

- **Level 1:** a first hash function divides the n keys into n buckets (like normal hashing with chaining).
- **Level 2:** for a bucket having n_j keys, a second hash table of size n_j^2 with its own hash function is built, chosen so that there is no collision.
- Total memory used is O(n) and a search needs only two hash computations, so it is O(1) always.

3. Sorting Techniques

Sorting means arranging the elements in ascending or descending order. Sorted data makes searching (e.g. binary search) faster. Important terms:

- **Stable sort:** equal elements keep their original relative order (bubble, insertion, merge are stable).
- **In-place sort:** needs only a constant amount of extra memory (bubble, selection, insertion).
- **Time complexity** is measured by the number of comparisons/swaps as a function of n.

3.1 Bubble sort

Bubble sort repeatedly **compares two adjacent elements and swaps them if they are in the wrong order**. In each pass the largest remaining element "bubbles up" to the end of the list. For n elements, n-1 passes are needed.

Algorithm: BUBBLE_SORT(A, n)

Step 1: Repeat Steps 2 and 3 for pass = 1 to n - 1.

Step 2: Repeat for j = 0 to n - pass - 1 :

Step 3: if A[j] > A[j+1] then swap A[j] and A[j+1].

Step 4: Stop. (Optimisation: if no swap happens in a pass, the list is already sorted, so stop early.)

Example: sort [5, 1, 4, 2, 8]

Pass	Comparisons and swaps	List after the pass
1	(5,1) swap -> 1 5 4 2 8; (5,4) swap -> 1 4 5 2 8; (5,2) swap -> 1 4 2 5 8; (5,8) no swap	1 4 2 5 **8**
2	(1,4) no swap; (4,2) swap -> 1 2 4 5 8; (4,5) no swap	1 2 4 **5 8**
3	(1,2) no swap; (2,4) no swap -> no swap in this pass, so the list is sorted	**1 2 4 5 8**

```
void bubble_sort(int a[], int n) {
    int i, j, t, swapped;
    for (i = 0; i < n - 1; i++) {
        swapped = 0;
        for (j = 0; j < n - i - 1; j++)
            if (a[j] > a[j + 1]) { t = a[j]; a[j] = a[j+1]; a[j+1] = t; swapped = 1; }
        if (swapped == 0) break;       // already sorted
    }
}
```

Complexity: best case O(n) (already sorted, with the swapped flag), average and worst case O(n^2). Space O(1). Stable.

3.2 Selection sort

Selection sort repeatedly **selects the smallest element** from the unsorted part and **places it at the beginning** of the unsorted part (swaps it with the first unsorted element). The sorted part grows from left to right.

Algorithm: SELECTION_SORT(A, n)

Step 1: Repeat Steps 2 to 4 for i = 0 to n - 2.

Step 2: Assume min = i.

Step 3: For j = i + 1 to n - 1 : if A[j] < A[min] then min = j.

Step 4: Swap A[i] and A[min].

Step 5: Stop.

Example: sort [64, 25, 12, 22, 11]

Pass	Minimum of unsorted part	List after swap (sorted part | unsorted part)
1	11	11 | 25 12 22 64
2	12	11 12 | 25 22 64
3	22	11 12 22 | 25 64
4	25	11 12 22 25 | 64
Result		11 12 22 25 64

```
void selection_sort(int a[], int n) {
    int i, j, min, t;
    for (i = 0; i < n - 1; i++) {
        min = i;
        for (j = i + 1; j < n; j++)
            if (a[j] < a[min]) min = j;
        t = a[i]; a[i] = a[min]; a[min] = t;
    }
}
```

Complexity: best, average and worst case all $O(n^2)$ (always scans the whole unsorted part). Space O(1). Makes fewer swaps (at most n-1) than bubble sort. Not stable.

Bubble Sort	Selection Sort
Swaps adjacent elements if out of order	Selects the minimum and swaps it to its correct position
Many swaps	At most n - 1 swaps
Best case O(n) (with flag)	Always $O(n^2)$
Stable	Not stable (normally)
Largest element goes to the end in each pass	Smallest element goes to the front in each pass

3.3 Insertion sort

Insertion sort builds the sorted list one element at a time, like arranging playing cards in your hand. Each new element is **inserted into its correct place** in the already sorted part by shifting larger elements to the right.

Algorithm: INSERTION_SORT(A, n)

Step 1: For i = 1 to n - 1 repeat Steps 2 to 5.

Step 2: key = A[i], j = i - 1.

Step 3: While j >= 0 and A[j] > key : A[j+1] = A[j] ; j = j - 1.

Step 4: A[j+1] = key.

Step 5: Stop.

Example: sort [12, 11, 13, 5, 6]

Pass	Key	List after the pass (sorted part | rest)
1	11	11 12 | 13 5 6
2	13	11 12 13 | 5 6
3	5	5 11 12 13 | 6
4	6	5 6 11 12 13

```
void insertion_sort(int a[], int n) {
    int i, j, key;
    for (i = 1; i < n; i++) {
        key = a[i]; j = i - 1;
        while (j >= 0 && a[j] > key) { a[j + 1] = a[j]; j--; }
        a[j + 1] = key;
    }
}
```

Complexity: best case O(n) (already sorted), average and worst case $O(n^2)$. Space O(1). Stable. Very good for small or nearly sorted data.

3.4 Merge sort

Merge sort is a **divide and conquer** algorithm: **divide** the list into two halves, **sort** each half recursively, and then **merge** the two sorted halves into one sorted list.

Algorithm: MERGE_SORT(A, low, high)
Step 1: If low < high : mid = (low + high) / 2.
Step 2: MERGE_SORT(A, low, mid) - sort the left half.
Step 3: MERGE_SORT(A, mid + 1, high) - sort the right half.
Step 4: MERGE(A, low, mid, high) - merge the two sorted halves.

Merging two sorted lists: compare the first elements of both lists, copy the smaller one into the result and move ahead in that list. Continue until one list ends, then copy the rest of the other list.

Example: sort [38, 27, 43, 3, 9, 82, 10]

Phase	Lists
Divide	[38 27 43 3 9 82 10] -> [38 27 43 3] and [9 82 10]
Divide	[38 27] [43 3] and [9] [82 10]
Divide	[38] [27] [43] [3] and [9] [82] [10]
Merge	[27 38] [3 43] and [9] [10 82]
Merge	[3 27 38 43] and [9 10 82]
Final merge	**[3 9 10 27 38 43 82]**

```
void merge(int a[], int low, int mid, int high) {
    int b[100], i = low, j = mid + 1, k = low;
    while (i <= mid && j <= high)
        b[k++] = (a[i] <= a[j]) ? a[i++] : a[j++];
    while (i <= mid)  b[k++] = a[i++];
    while (j <= high) b[k++] = a[j++];
    for (k = low; k <= high; k++) a[k] = b[k];
}
void merge_sort(int a[], int low, int high) {
    if (low < high) {
        int mid = (low + high) / 2;
        merge_sort(a, low, mid);
        merge_sort(a, mid + 1, high);
        merge(a, low, mid, high);
    }
}
```

Complexity: O(n log n) in best, average and worst cases. Needs extra space O(n) for merging. Stable. Not in-place.

3.5 Comparison of sorting techniques (time complexity analysis)

Algorithm	Best case	Average case	Worst case	Extra space	Stable?
Bubble sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes
Selection sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	No
Insertion sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes
Merge sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes
Quick sort (CO2)	O(n log n)	O(n log n)	$O(n^2)$	O(log n)	No
Heap sort (CO3)	O(n log n)	O(n log n)	O(n log n)	O(1)	No

How to remember

- $O(n^2)$ sorts (bubble, selection, insertion) are simple and good for small lists.
- O(n log n) sorts (merge, quick, heap) are used for large lists.
- Searching: linear O(n), binary O(log n), hashing O(1) on average.

4. Quick Revision - CO4

Remember these points

- Linear search checks one by one - O(n), works on unsorted data. Binary search halves a **sorted** list - O(log n).
- Hashing: h(k) = k mod m gives the index. Collision = two keys, same index.
- Collision resolution: chaining, or open addressing (linear: h+i, quadratic: $h+i^2$, double hashing).
- Direct-address table T[k] - no hash function needed. Perfect hashing - no collisions, static key set.
- Bubble: swap adjacent. Selection: pick the minimum. Insertion: insert in sorted part. Merge: divide and merge, O(n log n).

Practice questions

1. Explain linear search and binary search with an example and compare them.
2. Write an algorithm for binary search. Search 55 in 11, 22, 33, 44, 55, 66, 77.
3. What is a collision? Explain linear probing and quadratic probing with examples.
4. Explain direct-address tables and perfect hashing.
5. Insert 23, 43, 13, 27, 37 into a table of size 10 using linear probing.
6. Explain bubble sort and selection sort with an example and algorithm.
7. Sort 12, 11, 13, 5, 6 using insertion sort.
8. Explain merge sort with an example. Compare the time complexities of the sorting methods.